

CSDS 2026 – HW 3

Remark: Using AI to refine plots and RMarkdown formatting, and to improve the grammar of written answers.

Initialization

```
set.seed(20260311)
library(ggplot2)

knitr::opts_chunk$set(
  echo = TRUE,
  warning = FALSE,
  message = FALSE,
  fig.pos = "H",
  fig.align = "center",
  fig.show = "hold",
  dev.args = list(pointsize = 9)
)
```

Question 1

Let's reexamine how to *standardize* data: subtract the mean of a vector from all its values, and divide this difference by the standard deviation to get a vector of standardized values.

(a) Create a normal distribution (mean=940, sd=190) and standardize it to `rnorm_std`

```
rnorm_data <- rnorm(n = 1000, mean = 940, sd = 190)
rnorm_std <- scale(rnorm_data)
```

i. What should we expect the mean and standard deviation of `rnorm_std` to be, and why?

We expect the mean of `rnorm_std` to be 0 and the standard deviation to be 1, because standardization centers the data at 0 and scales it to unit variance.

And it did happen as expected:

```
mean_std <- mean(rnorm_std)
sd_std <- sd(rnorm_std)
cat("Mean:", mean_std, "\nStandard Deviation:", sd_std)

## Mean: -9.736656e-17
## Standard Deviation: 1
```

ii. What should the distribution (shape) of `rnorm_std` look like, and why?

The distribution of `rnorm_std` should look like a standard normal distribution (bell-shaped curve) because standardization does not change the shape of the distribution, it only shifts and scales it.

```
# --- predefine format for the plot ---
theme_set(theme_minimal())
```

```

base_theme <- theme(
  plot.title = element_text(hjust = 0.5),
  legend.position = c(0.99, 0.99),
  legend.justification = c(1, 1),
  legend.title = element_blank()
)

plot_density <- function(
  x,
  title,
  x_label,
  fill_color = "#cee9f2",
  line_color = "#3e5ed2",
  show_hist = TRUE
) {
  p <- ggplot(data.frame(value = x), aes(x = .data[["value"]]))

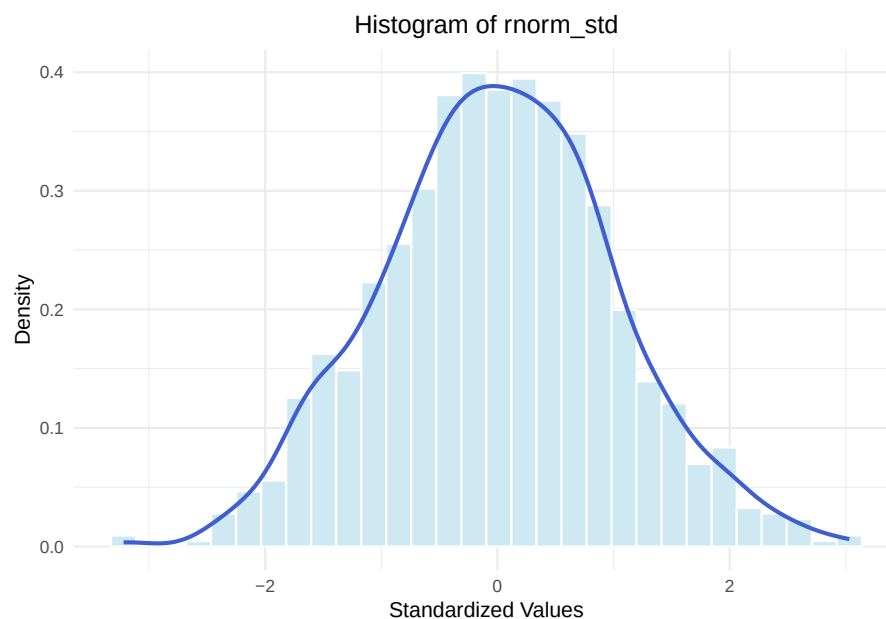
  if (show_hist) {
    p <- p + geom_histogram(
      aes(y = after_stat(density)), fill = fill_color, color = "white"
    )
  }

  p + geom_density(color = line_color, linewidth = 1) +
  labs(title = title, x = x_label, y = "Density") + base_theme
}

# -----

plot_density(
  x = rnorm_std,
  title = "Histogram of rnorm_std",
  x_label = "Standardized Values"
)

```



iii. What do we generally call distributions that are normal and standardized?

We generally call them “standard normal distribution” or “Z-distribution”, which has a mean of 0 and a standard deviation of 1.

(b) Create a standardized version of minday discussed in question 3 (let’s call it minday_std)

```
# minday from question 3
bookings <- read.table("first_bookings_datetime_sample.txt", header = TRUE)
hours <- as.POSIXlt(bookings$datetime, format = "%m/%d/%Y %H:%M")$hour
mins <- as.POSIXlt(bookings$datetime, format = "%m/%d/%Y %H:%M")$min
minday <- hours * 60 + mins

# create minday_std
minday_std <- scale(minday)
```

i. What should we expect the mean and standard deviation of minday_std to be, and why?

We should expect the mean of minday_std to be 0 and the standard deviation to be 1, because standardization centers the data at 0 and scales it to unit variance, same as with rnorm_std.

And it did happen as expected:

```
mean_minday_std <- mean(minday_std)
sd_minday_std <- sd(minday_std)
cat("Mean:", mean_minday_std, "\nStandard Deviation:", sd_minday_std)

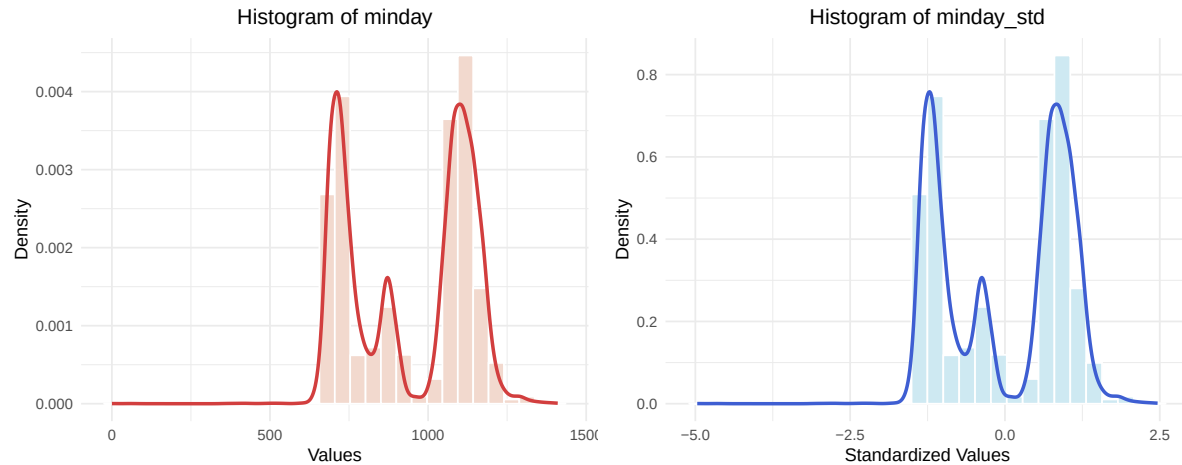
## Mean: -6.60989e-16
## Standard Deviation: 1
```

ii. What should the distribution of minday_std look like compared to minday, and why?

The distribution of minday_std should have the same shape as minday, including skewness, peaks, and other characteristics, but shifted to have a mean of 0 and scaled to have a standard deviation of 1.

```
plot_density(
  x = minday, title = "Histogram of minday",
  x_label = "Values", fill_color = "#f2d9ce", line_color = "#d23e3e"
)

plot_density(
  x = minday_std, title = "Histogram of minday_std",
  x_label = "Standardized Values"
)
```



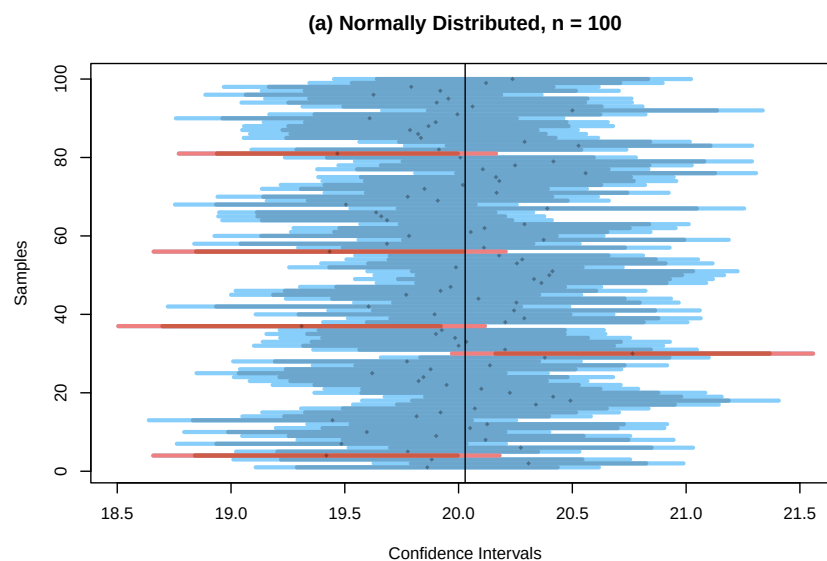
Question 2

Install the “compstatslib” package from Github (see week 2 class handout) and run the `plot_sample_ci()` function that simulates samples drawn randomly from a population.

(a) Simulate 100 samples (each of size 100), from a normally distributed population of 10,000

```
library(compstatslib)

plot_sample_ci(num_samples = 100, sample_size = 100, pop_size = 10000,
               distr_func = rnorm, mean = 20, sd = 3)
title(main = "(a) Normally Distributed, n = 100")
```



i. How many samples do we expect to NOT include the population mean in its 95% CI?

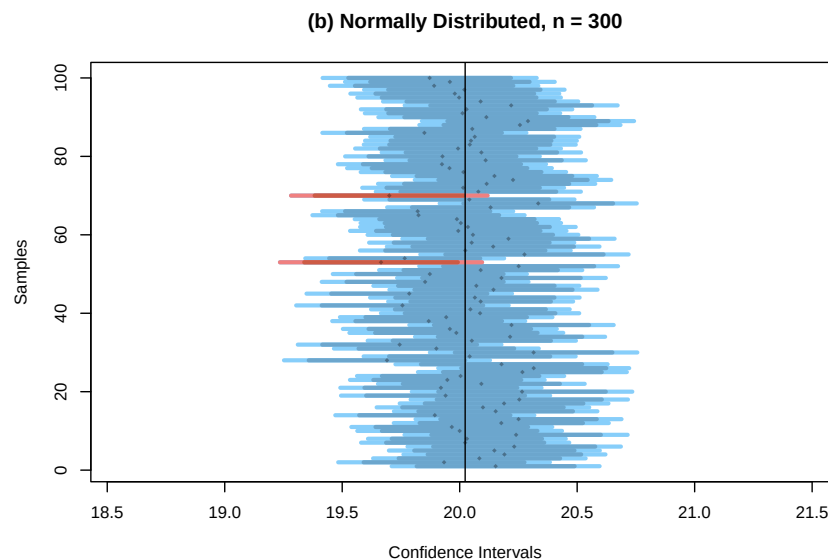
We expect about $100 * 0.05 = 5$ samples to not include the population mean in their 95% confidence intervals.

- ii. **How many samples do we expect to NOT include the population mean in their 99% CI?**

We expect about $100 * 0.01 = 1$ sample to not include the population mean in their 99% confidence intervals.

- (b) Rerun the previous simulation with the same number of samples, but larger sample size (`sample_size=300`):

```
plot_sample_ci(num_samples = 100, sample_size = 300, pop_size = 10000,
               distr_func = rnorm, mean = 20, sd = 3)
title(main = "(b) Normally Distributed, n = 300")
```



- i. **Now that the size of each sample has increased, do we expect their 95% and 99% CI to become wider or narrower than before?**

The confidence interval is given by Eq. (1).

$$\bar{x} \pm z \frac{\sigma}{\sqrt{n}} \quad (1)$$

As the sample size n increases, the standard error $\frac{\sigma}{\sqrt{n}}$ becomes smaller. So both the 95% and 99% confidence intervals become **narrower** than before.

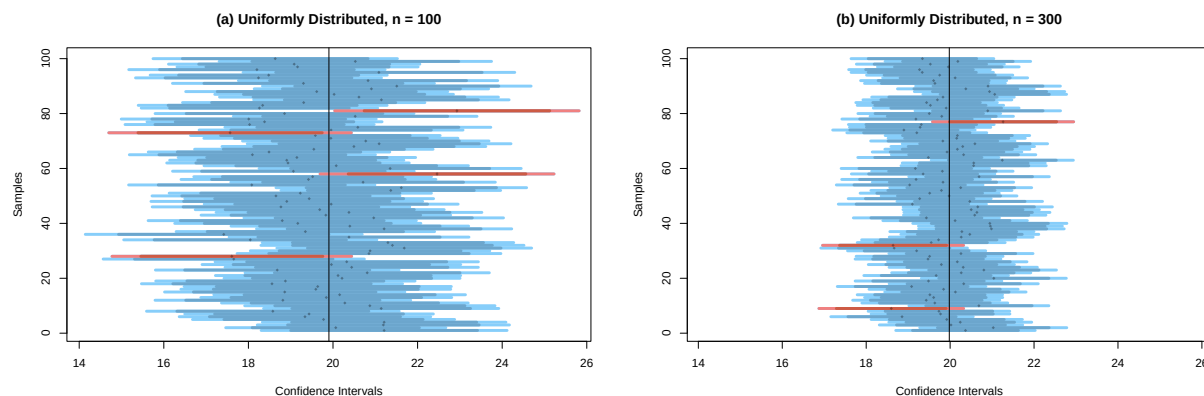
- ii. **This time, how many samples (out of the 100) would we expect to NOT include the population mean in its 95% CI?**

We still expect about $100 * 0.05 = 5$ samples to not include the population mean in their 95% confidence intervals, because the confidence level (95%) has not changed.

(c) If we ran the above two examples (a and b) using a uniformly distributed population, how do you *expect* your answers to (a) and (b) to change, and *why*?

```
# (a)
plot_sample_ci(num_samples = 100, sample_size = 100, pop_size = 10000,
               distr_func = runif, min = 0, max = 40)
title(main = "(a) Uniformly Distributed, n = 100")

# (b)
plot_sample_ci(num_samples = 100, sample_size = 300, pop_size = 10000,
               distr_func = runif, min = 0, max = 40)
title(main = "(b) Uniformly Distributed, n = 300")
```



Compared with the case of *normally distributed* population, the expected miss counts are still about the same, with around 5 out of 100 for 95% CI and 1 out of 100 for 99% CI. Although `rnorm` generates bell-shaped data and `runif` generates flat, equally likely data, CI coverage is determined by the confidence level, so changing the population shape does not change the target coverage.

Question 3

The company EZTABLE has an online restaurant reservation platform that is accessible by mobile and web. Imagine that EZTABLE would like to start a promotion for new members to make their bookings earlier in the day.

A *sample* data about their new members: the date and time for which they make their first ever booking using EZTABLE.

(a) What is the *average* booking time for new members making their first restaurant booking?

Note: use `minday`, which is the absolute minute of the day from 0-1440.

- i. Use traditional statistical methods to estimate the population mean of `minday`, its *standard error*, and the *95% confidence interval* (CI) of the sampling means

```
# We've loaded the data in Q1(b), so we can directly use `minday` here.
minday_mean <- mean(minday)
minday_se <- sd(minday) / sqrt(length(minday))
```

```
minday_ci_95 <- minday_mean + c(-1.96, 1.96) * minday_se
cat("Mean:", minday_mean, "\nStandard Error:", minday_se,
    "\n95% CI:", round(minday_ci_95[1], 2), " ~ ", round(minday_ci_95[2], 2))

## Mean: 942.4963
## Standard Error: 0.5997673
## 95% CI: 941.32 ~ 943.67
```

ii. Bootstrap to produce 2000 new samples from the original sample

```
resamples <- replicate(2000, sample(minday, length(minday), replace = TRUE))
cat("Resamples Dimension:", dim(resamples))

## Resamples Dimension: 100000 2000
```

iii. Visualize the means of the 2000 minday bootstrapped samples

```
resampled_means <- colMeans(resamples)
cat("Mean of Bootstrap Sample Means:", mean(resampled_means))

## Mean of Bootstrap Sample Means: 942.4984
```

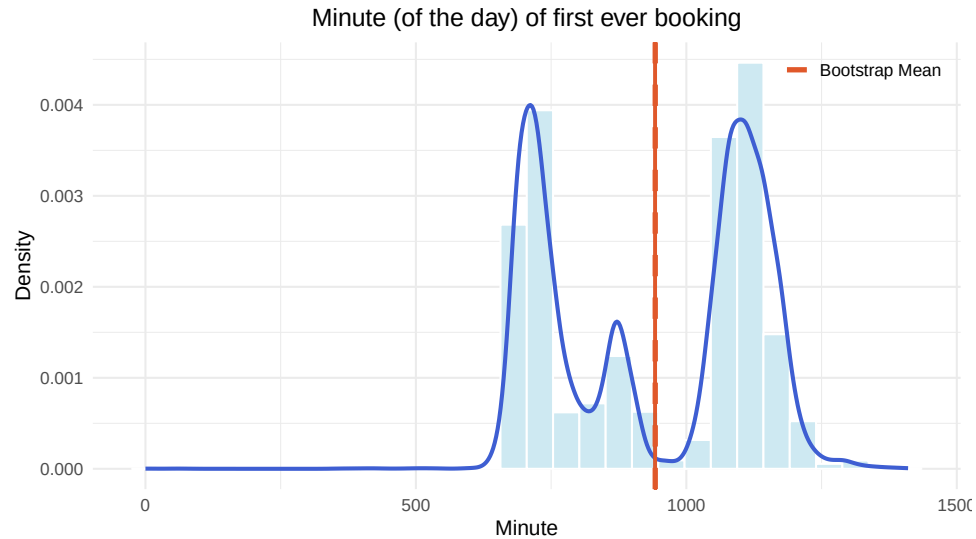
We can view the means of the 2000 bootstrapped samples in 2 ways:

- (1) Showing all 2000 bootstrapped sample means as vertical lines, highlighting their average, and comparing it with the original sample mean on the minday density plot.

```
# ----- predefine function for the plot -----
plot_sampling_center_lines <- function(
  x,
  centers,
  title,
  center_label,
  x_label = "Value",
  line_color = "#e0582a"
) {
  plot_density(x = x, title = title, x_label = x_label) +
    geom_vline(
      aes(xintercept = mean(centers), color = center_label),
      linewidth = 1.3, linetype = "dashed",
      key_glyph = "path"
    ) +
    geom_vline(
      xintercept = centers, color = line_color,
      linewidth = 0.3, alpha = 0.25
    ) +
    scale_color_manual(values = setNames(line_color, center_label)) +
    base_theme
}

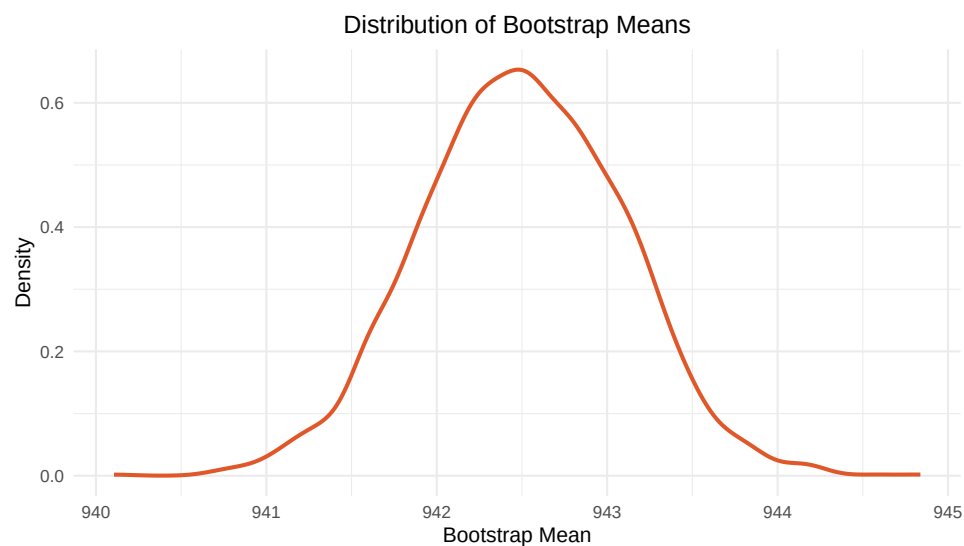
# -----

plot_sampling_center_lines(
  x = minday,
  centers = resampled_means,
  center_label = "Bootstrap Mean",
  title = "Minute (of the day) of first ever booking",
  x_label = "Minute"
)
```



(2) Simply observe the distribution of the bootstrapped means.

```
plot_density(
  x = resampled_means,
  title = "Distribution of Bootstrap Means",
  x_label = "Bootstrap Mean",
  show_hist = FALSE,
  line_color = "#e0582a"
)
```



iv. Estimate the **95% CI** of the bootstrapped means using the quantile function

```
quantile(resampled_means, probs = c(0.025, 0.975))
##      2.5%    97.5%
## 941.3265 943.6450
```


(b) By what time of day, have half the new members of the day already arrived at their restaurant?

i. Estimate the *median* of minday

```
minday_median <- median(minday)
cat("Median:", minday_median)

## Median: 1040
```

ii. Visualize the *medians* of the 2000 bootstrapped samples

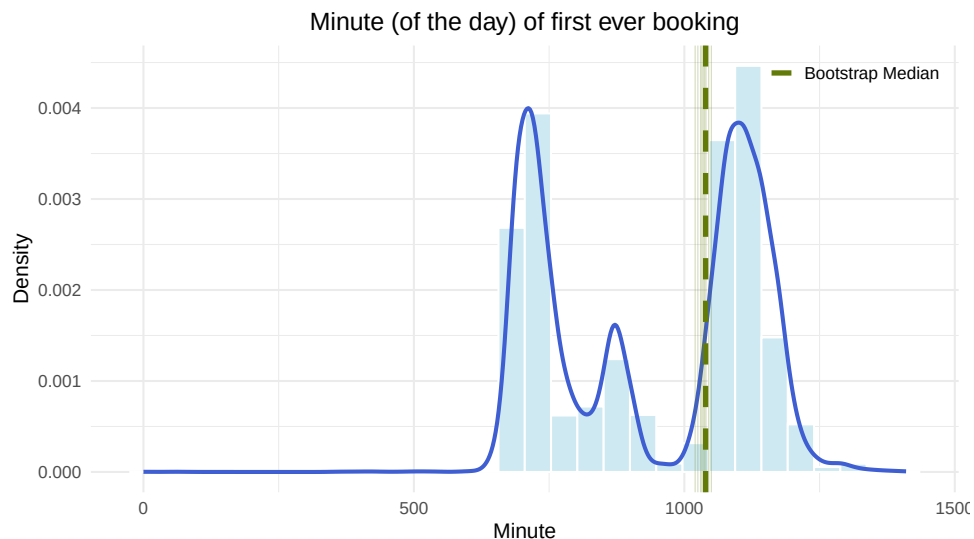
```
resampled_medians <- apply(resamples, 2, median)
cat("Mean of Bootstrap Sample Medians:", mean(resampled_medians))

## Mean of Bootstrap Sample Medians: 1039.355
```

We can view the medians of the 2000 bootstrapped samples in 2 ways:

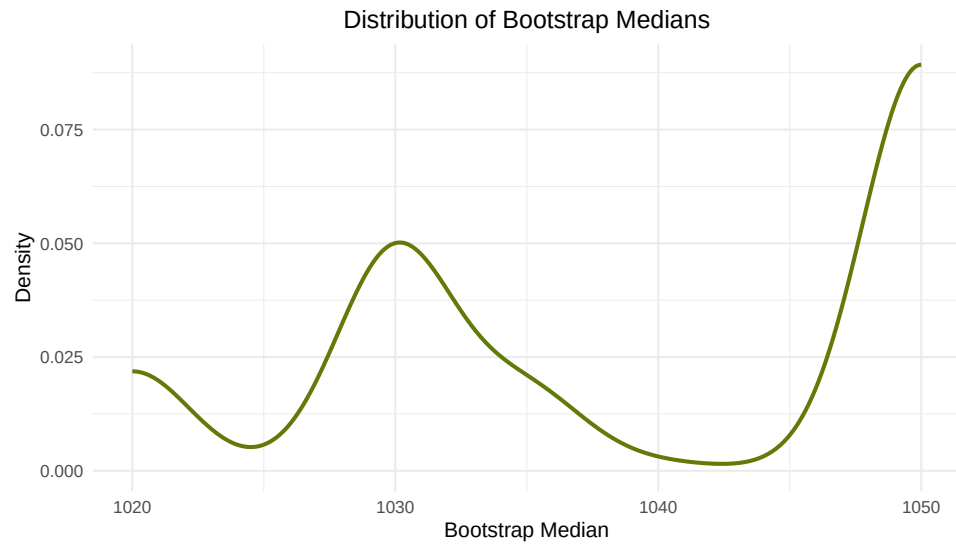
- (1) Showing all 2000 bootstrapped sample medians as vertical lines, highlighting their average, and comparing it with the original sample median on the minday density plot.

```
plot_sampling_center_lines(
  x = minday,
  centers = resampled_medians,
  center_label = "Bootstrap Median",
  title = "Minute (of the day) of first ever booking",
  x_label = "Minute",
  line_color = "#627b08"
)
```



- (2) Simply observe the distribution of the bootstrapped medians.

```
plot_density(
  x = resampled_medians,
  title = "Distribution of Bootstrap Medians",
  x_label = "Bootstrap Median",
  show_hist = FALSE,
  line_color = "#627b08"
)
```



iii. Estimate the *95% CI* of the bootstrapped medians using the quantile function

```
quantile(resampled_medians, probs = c(0.025, 0.975))
```

```
## 2.5% 97.5%
```

```
## 1020 1050
```